M2 - Measurement Guide

M2 Measurement Guide — Baseline Performance

CSCE 553 · Fall 2026

Use this with **M2 — Baseline Performance**.

You will: pick parameters → pick tools → design a stair-step → run it → save CSVs and resource notes → write the report.

The supported driver is a **Python 3.12+ stdlib script** (below). `hey`, Locust, and k6 are allowed if they emit the same numbers. You still need resource signals; a load-tool screenshot is not enough.

---

0. What “done” looks like

A teammate can:

1. Read `m2/m2-report.md` and see a **prediction dated before** the sweep.
2. Re-run `python load_baseline.py ...` against your hosted URL and localhost.
3. Open `m2/results/*.csv` and rebuild the three required charts.
4. Point to CPU / memory / connections (or disk) **at the same clock time** as a load stage.

Architecture stays: load generator → API → Postgres. No new products.

---

1. Performance parameters

Back to Top

These are the only numbers the report must treat as first-class. Use the names consistently in CSV headers and figures.

1.1 Load (independent variable)

Name	Meaning	How you set it
Offered concurrency *N*	Number of clients in flight	`--concurrency` on the script (closed-loop: each client sends the next request when the last returns)
Duration	Steady-state window	10 s warmup + 60 s measure
Mix	Fraction of reads vs writes	Separate runs, or `--mix 0.8`
Payload	Bytes in typical response	Note from one `curl -w "%{size_download}"`

Closed-loop concurrency is what this script does. It is **not** the same as "N users on the website." Convert in the worksheet: RPS ≈ active users × actions / seconds.

Little's Law (check your own table): *L* ≈ throughput × mean latency. If you measured 40 successful req/s at 0.25 s mean latency, expect ~10 requests in flight. If concurrency was 16 and *L* is 10, clients were idle or the server was rejecting work — say so.

1.2 Service behavior (dependent variables)

Name	Definition	Do not substitute
Throughput	Count of **HTTP 2xx/3xx** in the measure window ÷ wall seconds	Do not count 5xx as successful RPS
p50 / p95 / p99	Latency percentiles of **completed** attempts in the window (success and fail, unless you document otherwise)	Do not report only the mean
Max latency	Worst completed attempt in the window	Useful for timeouts
Error rate	Attempts that are timeout, connection error, or HTTP ≥ 400 ÷ all attempts	Include 401 if your token expired — that is still an error for the test

Cold-start latency is a separate one-off measurement, not part of the percentile of a 60 s warm run. Mix them and p99 becomes "Render woke up."

1.3 Resource signals (the chain of evidence)

The lecture: load → service effect → resource signal, **aligned timestamps**.

Pick what you can actually see on your stack:

Signal	Hosted (typical M1)	Local API
API CPU / memory	Render → your web service → **Metrics**	Task Manager, Activity Monitor, or `docker stats` if you containerized
DB CPU / connections	Supabase → **Reports** / Database settings. SQL: `SELECT count(*) FROM pg_stat_activity;`	Same SQL against local Postgres
Disk I/O	Often invisible on free PaaS; say "not observable"	Windows Resource Monitor, `iostat`, Docker stats block I/O
Host sleep	First request after 15 min idle	Does not apply

You need **at least two** resource signals for the hosted stair-step. "The API got slow" without a resource is incomplete.

1.4 Objectives (derived, not raw)

An **SLI** is a user-facing signal (latency of the list endpoint; success ratio). An **SLO** is a threshold on that SLI. You propose one SLO from data in the report. You do not need Prometheus.

---

2. Tools

2.1 Supported: Python driver (no extra packages)

Copy `load_baseline.py` from **Section 7** into your app repo. It uses only the standard library.

```
python --version    # 3.12+ recommended; 3.11 is fine
python load_baseline.py --help
```

On Windows use the same venv as M1 if you want; the script does not need FastAPI installed.

2.2 Optional tools (allowed)

Tool	Strength	Use it only if
hey	One URL, easy histograms	You already have a token and a GET URL
Locust	Browser UI, mix of tasks	You export CSV / stats, not only a screenshot
k6	Scripted, industry-shaped	Same: JSON/CSV in `m2/results/`

`ab` (ApacheBench) is weak on HTTPS and tails. Prefer hey or the Python driver.

For **hey** after you have logged in (replace token and URL):

```
hey -n 2000 -c 8 -m GET -H "Authorization: Bearer YOUR_TOKEN" "https://YOUR.onrender.com/api/messages"
```

`-n` is total requests, `-c` is concurrency. Convert hey's report into the same table columns as the Python driver so the report is comparable.

2.3 Tools for resources

- **Clock:** Use one machine's clock for "stage started 15:42:10." Screenshot the dashboard with a visible time, or write the time in the filename: `render-cpu-H-c8-154210.png`.
- **curl:** Sanity check before every series (`GET /health`, then login).
- **SQL editor (Supabase):** paste the `pg_stat_activity` query between stages.
- **Spreadsheet or Python matplotlib/Excel:** plot CSV. Plots may be ugly. Label axes: offered concurrency; RPS or milliseconds.

2.4 What not to use as the only instrument

Browser DevTools Network tab (too few requests). Postman runner (hard to get percentiles). Hitting the **HTML pages** instead of `/api/...` (you will measure templates and static files, not the API you will scale later).

---

3. Designing the experiments

3.1 Write the model first

Fill the 10K / 1M / 100M worksheet in a file `m2/results/prediction.md` **before** the first 60 s run. Commit it, or paste a screenshot of the file with a timestamp. Then predict:

> First limit: _____ (e.g. Render CPU, DB connections, API memory, request queue in the framework).

A reasonable M1 guess: **one 0.1 CPU / 512 MB web process** saturates before **500 MB Postgres**, on the **read** path, unless you open a new DB connection per request (then **connections** may win). You must justify *your* guess from how your app uses the DB (pool vs connect-per-request). If you do not know, open the code and look; “AI wrote it” is not an assumption.

3.2 Factors you hold constant

For a given curve (throughput vs *N*):

- Same git commit
- Same endpoint and JSON body
- Same auth token (refresh if 401s appear — then note it)
- Same mix (100% read for the main sweep)
- Same region / laptop on AC power

Vary only **concurrency** (and environment H vs L as a **separate** curve).

3.3 Factor you do not hide

Render **free** web services sleep after ~15 minutes idle and take ~1 minute to wake. That is a **shape** effect (burst of latency), not your steady-state capacity. Design:

1. Measure cold start once (required).
2. `curl /health` until it is fast.
3. Only then start warmup + 60 s stages.

Series **L** answers: "If the host did not sleep and had my laptop's CPU, where is the knee?" Do not declare H "broken" because of sleep; declare sleep a **confound** for p99 unless you warmed up.

3.4 Endpoints (map from M1)

Product	Read path	Write path	Control
Social	`GET /api/messages`	`POST /api/messages` with a small JSON body	`GET /health`
E-commerce	`GET /api/products` or `GET /api/products?q=`	`POST /api/cart/items` or `PATCH` quantity	`GET /health`
Existing app	Your documented list/search	Your documented create/update	Health

Login once per series:

```
curl -sS -X POST "$BASE/api/auth/login" \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"alex@example.com\",\"password\":\"ClassDemo123!\"}"
```

Put the token in `M2_TOKEN` (environment variable), never in git.

3.5 Stop rules and ethics

You are loading **your** class copy, not the university, not Render as a whole. Still:

- Cap hosted concurrency at **32** unless the instructor says otherwise.
- 60 s measure + 10 s warmup per stage; a full H+L read sweep is under ~20 minutes of load.
- If you see **pause**, **billing**, or **connection refused** from Supabase, stop. Resume the project; shorten the remaining stages.
- Never point a load test at a URL you do not own.

3.6 Repeatability

Repeat the last **healthy** stage. If RPS differs by more than ~20%, look for: sleep mid-test, token expiry, laptop thermal throttle, someone else using the DB. Report both runs; do not silently pick the prettier one.

---

4. Executing the experiments

Do this in order. Do not start Locust “to see what happens” before the prediction file exists.

4.1 Prep (both environments)

1. Confirm M1 still works: hosted `curl` health + login + one read (same as M1 evidence).
2. **Seed the read-path table** if the list/search response is empty or tiny (`[]`, a handful of rows). An empty database is not the workload you are modeling. Copy `seed_data.py` from the handout pack and POST enough rows that `curl -w "%{size_download}"` on the **read** path is stable and non-trivial (on the order of **50** posts or products). Use the same write API the app already has; this is sequential inserts, not a load test. Re-running the script **adds** more rows — run it once, then leave the table alone for the read sweep.
3. Create `m2/results/` in the repo. Add `m2/results/*.png` to git if you want screenshots; keep `.env` gitignored.
4. Write `m2/results/prediction.md`.
5. Open Render metrics and Supabase reports in a browser; leave them open.

4.2 Series H (hosted)

```
# 1. Cold start (only if the service has been idle ≥ 15 min)
curl -sS -o NUL -w "cold_ms=%{time_total}\n" https://YOUR.onrender.com/health

# 2. Warm
curl -sS https://YOUR.onrender.com/health

# 3. Login; export token
# 4. Control
python load_baseline.py --base https://YOUR.onrender.com --path /health --no-auth --concurrency 2 --duration
60 --label H-health-c2

# 5. Read sweep
python load_baseline.py --base https://YOUR.onrender.com --path /api/messages --token %M2_TOKEN% --
concurrency 1 --duration 60 --label H-read-c1
python load_baseline.py --base https://YOUR.onrender.com --path /api/messages --token %M2_TOKEN% --
concurrency 4 --duration 60 --label H-read-c4
# ... 8, 16, (32 if still healthy)

# 6. Write (one stage near the read knee), then mix
python load_baseline.py --base https://YOUR.onrender.com --path /api/messages --token %M2_TOKEN% --method
POST --body "{\"body\":\"m2 load\"}" --concurrency 8 --duration 60 --label H-write-c8
python load_baseline.py --base https://YOUR.onrender.com --path /api/messages --token %M2_TOKEN% --mix 0.8 --
write-path /api/messages --write-body "{\"body\":\"m2 load\"}" --concurrency 8 --duration 60 --label H-mix-c8
```

On macOS/Linux use `$M2_TOKEN` and `/dev/null` instead of `NUL`. Adjust paths for e-commerce.

Between stages: wait 30–60 s. Glance at `pg_stat_activity` and Render CPU. Screenshot if the needle moved.

During a 60 s run: do not browse the app in another tab on the same instance if you can avoid it (extra load). Do snapshot resources once in the middle of the stage.

4.3 Series L (local API, same database)

Same commit. From the M1 README:

```
# venv on, .env has DATABASE_URL to the same Supabase
uvicorn main:app --host 127.0.0.1 --port 8000
```

Point the driver at `http://127.0.0.1:8000`. Repeat health control + read sweep (skip cold start). You are isolating **hosting**, not rewriting the app.

If local cannot reach Supabase (firewall, IPv6), say so and use a local Postgres with a **restored schema and seed** ☁then series L is not comparable on DB latency; put that in confounds.

4.4 If a stage explodes

Stop. Record the error rate and p99. That row is the **measured limit**. You do not need a perfect curve on the far side of saturation. Recover (wait), repeat the previous stage once, then move to writes/mix if you still can, or stop hosted load if the project is unhealthy.

4.5 Windows notes

- Run the driver from **PowerShell** or `cmd` in the repo. Use `python`, not the Store stub, if `python` opens the Microsoft Store — `py -3 load_baseline.py`.
- Set token: `$env:M2_TOKEN = "..."` then `--token $env:M2_TOKEN`.
- Laptop on **power**, not battery, for series L.

---

5. Collecting data

5.1 Files

File	Contents
`m2/results/prediction.md`	Worksheet + first-limit sentence
`m2/results/summary.csv`	One row per stage (the driver appends this)
`m2/results/H-read-c8.requests.csv`	Optional per-request latencies if you pass `--raw`
`m2/results/notes.md`	Clock times, screenshots names, odd events (pause, 401, sleep)
Screenshots	Render CPU/mem, Supabase, Task Manager; filenames include series and concurrency

`summary.csv` columns (driver default):

```
label,env_note,concurrency,duration_s,attempts,successes,errors,rps_success,p50_ms,p95_ms,p99_ms,max_ms,error
_rate
```

5.2 Resource log (hand-written is fine)

Example row in `notes.md`:

```
15:47:02  H-read-c8 start
15:47:35  Render CPU ~45% (screenshot render-H-c8-154735.png)
15:47:36  pg_stat_activity count=12
15:48:02  H-read-c8 end  rps=... p99=...
```

That is the **chain of evidence**. Fancy observability is a later milestone.

5.3 What to keep out of git

Tokens, `DATABASE_URL`, `.env`, full HAR files with cookies. CSVs of latencies are fine. If a screenshot shows a secret, crop it.

5.4 Plotting

Three figures, **separate series H and L** (two lines or two panels — not averaged):

1. Successful RPS vs concurrency
2. p50 and p99 (ms) vs concurrency
3. Error rate vs concurrency

Mark the **knee** (last stage before RPS flattens or p99/errors explode). Title each figure with endpoint and environment. Caption: tool, duration, commit hash.

Python snippet (optional, needs matplotlib only if you want):

```
# plot_m2.py — optional; Excel is acceptable
import csv
from collections import defaultdict

# Read summary.csv, filter label startswith H-read / L-read, plot.
```

You may paste spreadsheet charts as images into `m2/m2-report.md`.

---

6. Presenting

`m2/m2-report.md` is a **measurement paper**, not a diary. Suggested headings (match the assignment rubric):

1. **System under test** — one diagram, commit, hosted URL, local command.
2. **Workload model** — assumptions; 10K / 1M / 100M table; **predicted** first limit.
3. **Method** — driver and version (`python --version`), endpoints, stages, stop rules, H vs L.
4. **Results** — tables + three figures. Cold-start row. Mix/write if they differ from read.
5. **Resources** — aligned notes; which signal moved with p99.
6. **Claim** — one bottleneck. Use mix if writes saturated something else.
7. **SLO** — eligible events, *T*, *E*, pass/fail on H at the last healthy stage.
8. **Revise** — gap vs prediction; what the 1M and 100M columns would require (more CPU, pooling, cache, …) **without implementing them**.
9. **Confounds** — sleep, shared tenancy, laptop, clock, token expiry.
10. **Repro** — exact commands.

Tone: numbers, then interpretation. “p99 rose from 80 ms at *N*=4 to 2.1 s at *N*=16 while Render CPU sat at ~90% and DB connections stayed at 8” is a claim. “The cloud is slow” is not.

Little’s Law paragraph: pick one healthy stage, show $L \approx \text{RPS} \times \text{mean latency}$, compare to concurrency.

Do not recommend buying Render Pro as the finding. Cost is a system property: the free host **is** a limit. The next milestone is whether a **cache** (or index, or pool) moves the bottleneck — after this baseline exists.

---

7. Supported driver (copy into the repo)

Save the provided `load_baseline.py` at the repo root. Read and review it before you run it. You may need to customize it for your app.

Save the provided `login.py` at the repo root. It can be used to get the JWT token. You may need to customize it if your JSON uses another key, print `body` once and adapt.

Save the provided `seed_data.py` at the repo root. Use it in §4.1 to fill the table your **read** path lists (messages or products), not as a substitute for the 60 s write experiment. Adapt `--path` and `--body` to your README; `{n}` in the JSON becomes 1, 2, … so rows are unique.

Alternatively you can get the tokens using curl as in Section 3.4.

8. Troubleshooting

Symptom	What to try
Hosted p99 is ~60 s on the first stage	You measured sleep. Warm `/health`, then rerun. Keep the first number as cold start only.
All 401s	Token missing or expired; login again. `--no-auth` only for `/health`.
All 422/400 on writes	Body does not match your API; copy a working M1 curl.
Local fast, hosted dead	Expected gap: 0.1 CPU, sleep, extra RTT to Supabase. That *is* the comparison.
`pg_stat_activity` count stays 1	Connection pooling; CPU or the API event loop may still be the limit.
SSL errors local to Supabase	`sslmode=require` on `DATABASE_URL` (M1 guide).
Render suspends / hours exhausted	Stop hosted load. Finish remaining stages on series L; document it.
Script hangs	`--timeout 30`; lower concurrency; check you are not waiting on sleep without curl first.

9. What not to do

- Do not pay for hosting to flatten the hosted curve.
- Do not add Redis “just to have a cache number.” That is the next architecture change.
- Do not load-test the HTML UI as a substitute for the JSON API.
- Do not discard the run that hit the stop rule — it is the measured limit.

When prediction, CSVs, two environments, resource timestamps, and a bottleneck sentence exist, go back to the assignment submission list.

Last modified: Wednesday, September 2, 2026, 11:14 AM

◀ M2 - Inclass Prep

Jump to activity

M2 - Assignment ▶

UNIVERSITY of LOUISIANA LAFAYETTE

104 E. University Circle
Lafayette, LA 70503
(337) 482-1000

You are logged in as **Fupei Guo** (**Log out**)
Data retention summary
Get the mobile app